

Safe Drinking water Quality Screening using Concept learning and Decision trees.

1. Problem Statement:-

The objective is to develop an educational machine-learning System that Screens a water sample and classifies it into:

- Safe for use
- Needs Treatment

The System uses simple categorical water quality attributes such as pH band, turbidity, TDS band, residual chlorine, microbial indicators and source type.

The System demonstrates two approaches:-

1. Concept learning using Candidate Elimination / version space
2. Decision Tree learning using Entropy and Information Gain

1.2 learning problem:-

Let an instance be represented as:-

$$x = (\text{pH}, \text{Turbidity}, \text{TDS}, \text{Chlorine}, \text{Microbial}, \text{Source})$$

The target Concept is

$$C(x) = \begin{cases} \text{Safe for use} \\ \text{Needs treatment} \end{cases}$$

If sample satisfies the learned conditions otherwise

1.3 Attributes

Attributes	Possible values
pH Band	Acidic, neutral, Alkaline
Turbidity	low, medium, high
TDS Band	low, medium, high
Residual chlorine	low, normal, high
Microbial Indicators	Absent, present
Source Type	municipal, Ground water, Borewell, surface

1.4 Class labels

There are two target classes:

- Safe for use
- Need treatment

1.5 Hypothesis Representation:

We use a conjunctive hypothesis representation. Each hypothesis contains either:

- a specific attribute value or
- ?, meaning any value is acceptable

For example:-

$h = (\text{Neutral}, \text{low}, ?, \text{normal}, \text{Absent}, ?)$

2. learning workflow

water Sample Dataset



Define Attributes and target Concept



Select Training Examples



Initialize S and G Boundaries



Candidate Elimination



obtain Version Space Boundaries



Analyze Inductive Bias



Preprocessing / Encoding



Train - Test Split



Decision Tree using Entropy



Information Gain



Accuracy, Precision, Recall, F1



Compare CE and Decision Tree



Analyse Limitation + SDG6 + SDG3

3. Candidate elimination/version space.

Sno	PH	Toxicity	TDS	Chlorine	Microbial	Class.
1.	Neutral	low	low	Normal	Absent	Safe
2.	Neutral	low	low	normal	Absent	Safe
3.	Neutral	low	medium	normal	Absent	Safe
4.	Neutral	low	medium	normal	Absent	Safe
5.	Neutral	high	low	normal	Absent	Safe
6.	Alkaline	low	low	normal	Absent	need treatment
7.	neutral	low	low	high	Absent	need treatment
8.	neutral	low	low	low	Absent	need treatment
9.	neutral	low	low	normal	present	need treatment
10.	neutral	low	low	normal	Absent	Safe.

Candidate Elimination Boundary Table:-

Positive $\rightarrow$ $\langle \text{Neutral, low, low, normal, Absent, microbial} \rangle \rightarrow$
 $\langle ?, ?, ?, ?, ?, ? \rangle$

Negative $\rightarrow$ $\langle \text{Neutral, low, ?, normal, Absent, ?} \rangle \rightarrow$
 $\langle \text{Neutral, low?, normal, ?} \rangle$

Positive $\langle \text{Neutral, low?, normal, absent, ?} \rangle \rightarrow$
 $\langle \text{Neutral, low?, normal, Absent, ?} \rangle$

Conductive Bias

Inductive Bias is the set of assumption a machine-learning algorithm uses to generalize for limited training examples to unseen examples.

Example:- <Neutral, low, ?, Maximal, Absent, ?>

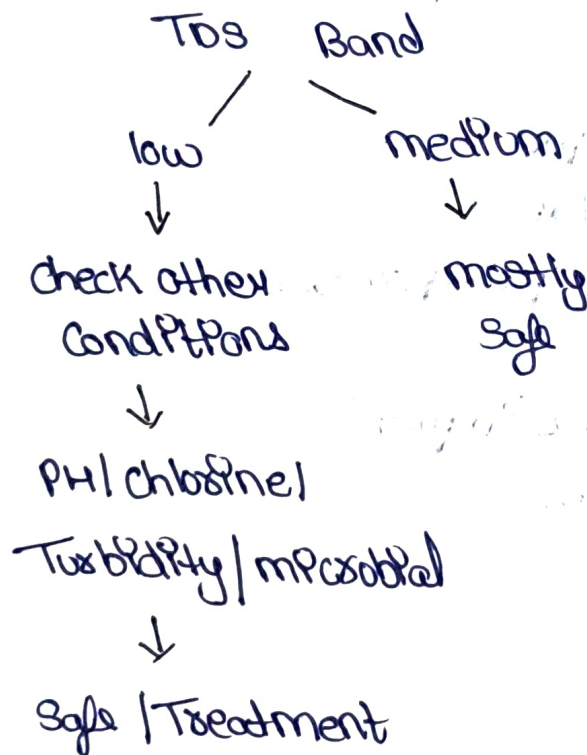
Decision Tree learning

The Decision Tree uses Entropy and Information Gain

Entropy - measures the impurity or uncertainty of a dataset

$$\text{Entropy}(S) = - \sum_{i=1}^C P_i \log_2(P_i)$$

Decision Tree Interpretation:



Dataset for python Implementation:-

for the computational experiment a synthetic dataset

Containing 200 records is generated

The dataset contains:

- 200 records
- 6 input attributes
- 2 output classes

Complete Google Colab python Code

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import
train_test_split

from sklearn.compose import

Column Transformer

from sklearn.tree import

DecisionTreeClassifier

from sklearn.tree import plot_tree
export_text

from sklearn.metrics import (

ConfusionMatrix

accuracy_score,

PrecisionScore

recall_score,

f1_score,

ClassificationReport

)

1. Generate Synthetic Dataset

```
rng = np.random.default_rng(42)
```

```
n = 200
```

```
status = []
```

```
labels = []
```

```
for i in range(n):
```

```
    if i < 100:
```

```
        row = [
```

```
            "Neutral"
```

```
            rng.choice(["low"])
```

```
            rng.choice(["low", "medium"]);
```

```
            "Normal"
```

```
            "Absent"
```

```
            rng.choice([
```

```
                "municipal"
```

```
                "Groundwater",
```

```
                "Borewell",
```

```
                "Surface"
```

```
            ])
```

```
    ]
```

```
    safe_condition = [
```

```
        row[0] = "Neutral"
```

```
        and row[1] = "low"
```

```
        and row[2] in ["low", "medium"]
```

```
and row[3] == "Normal"
```

```
and row[4] == "Absent"
```

```
)
```

```
while safe condition
```

```
labels = np.array(labels)
```

```
flip = np.random(n)/0.04
```

```
labels = np.where(
```

```
flip
```

```
np.where(
```

```
labels == "Safe for use",
```

```
"Need Treatment",
```

```
"Safe for use"
```

```
),
```

```
) labels
```

2. Create Data frame

```
df = pd.DataFrame(
```

```
    focus
```

```
    columns = [
```

```
        "PH Band",
```

```
        "Turbidity",
```

```
        "TDS Band",
```

```
        "Residual chlorine",
```

```
        "Microbial Indicators",
```

```
        "Source Type"
```

```
df["class"] = labels
```



```
Print ("Datatest shape")
```

```
Print (df.shape)
```

```
X = df.drop (columns = ["class"])
```

```
Y = df ["class"]
```

```
Categorical - features = X.columns.tolist()
```

```
Y_train, Y_test, X_train, X_test = train_test_split(
```

```
    X, Y, test_size = 0.20,
```

```
    random_state = 42
```

```
Preprocess = Column Transformer ([
```

```
    (cat, OneHotEncoder (handle_unknown = "ignore"),
```

```
    X.columns)
```

```
model = Pipeline ([
```

```
    ("preprocess", preprocess),
```

```
    ("tree", DecisionTreeClassifier(
```

```
        criterion = "entropy",
```

```
        max_depth = 6, random_state = 42])
```

```
model.fit (X_train, Y_train)
```

```
Y_pred = model.predict (Y_test)
```

```
Print ("Confusion matrix :\n", Confusion_matrix (Y_test, Y_pred))
```

```
Print ("Accuracy:", accuracy_score (Y_test, Y_pred))
```

```
Print ("Precision:", precision_score (Y_test, Y_pred, pos_label = "safe"))
```

```
Print ("Recall:", recall_score (Y_test, Y_pred, pos_label = "safe"))
```

```
Print ("F1-Score:", f1_score (Y_test, Y_pred, pos_label = "safe"))
```

Expected Model Results:-

metrics	Result
accuracy	0.9250
Precision	0.9048
Recall	0.9500
f1-score	0.9268

Confusion matrix

$$\begin{bmatrix} 19 & 1 \\ 2 & 18 \end{bmatrix}$$

Accuracy:-

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$TP = 19, TN = 18, FP = 2, FN = 1$$

$$\text{Accuracy} = \frac{19 + 18}{19 + 18 + 2 + 1} = \frac{37}{40}$$

$$\text{Accuracy} = 0.925 = 92.5\%$$

Precision:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{19}{19 + 2} = \frac{19}{21} = 90.48\%$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{19}{19 + 1} = \frac{19}{20} = 95\%$$

f1-Score:

$$f_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 0.9268 = 92.68\%$$

Clean water and Sanitation:

The model demonstrates how machine learning can assist in preliminary water-quality screening.

- faster preliminary screening
- Identification of sample that may need further testing
- Support for community level monitoring
- Improve accuracy of water quality

SDG 3 - Good health and well-being

- unsafe water can contribute to health problems
- A screening system can help identify samples that require further investigation before consumption
- Support preventive health practices by helping identify potentially problematic water samples.

False Safe and False Alert Risks

False Safe:

A Safe prediction occurs when
Actual - needs Treatment
Predicted = Safe for use

This can be dangerous because a user may incorrectly assume the water is safe

False Alert:-

Actual = safe for use

Predicted = needs Treatment

- unnecessary treatment → additional cost,
- inconvenience → unnecessary investigation

Limitations:-

- Dataset is Synthetic
- only a small no. of features are used
- water quality can change with location and season
- categorical bands may lose detailed information
- model accuracy may differ on real-world data.

Conclusion:-

This project shows how machine learning can be used to screen water quality. Candidate elimination helps identify the concept boundary while the Decision Tree uses Entropy and classifies water samples as safe or needs treatment. It is simple and easy to understand, but the dataset is synthetic and may not represent all real-world conditions. Overall, the project demonstrates the useful application of machine learning for clean & safe water screening.